

1. **Running JavaScript:** What JavaScript is and where it runs · The console: `console.log` and `console.assert` · Reading error messages · try and catch: handling errors
2. **Variables, types and values:** Declaring variables: `let`, `const` and `var` · Data types and `typeof` · Strings and template literals · Numbers, arithmetic and the remainder operator · Type coercion and `==` vs `===` · Truthy and falsy values
3. **Logic and control flow:** Logical operators and short-circuiting · Conditionals: `if/else`, the ternary and `switch` · Loops: `for`, `while`, `break` and `continue`
4. **Arrays and objects:** Arrays: indexes, `length`, `push` and `pop` · Objects: properties, dot and bracket notation · `for...of`, `for...in` and looping over strings · Values vs references: copying and mutation · JSON: data as text
5. **Functions and scope:** Functions: parameters, arguments and `return` · Scope, hoisting and the temporal dead zone · Declarations, expressions and arrow functions · Default parameters · Function scope and functions as values · Array methods: `forEach`, `map`, `filter`, `find`, `includes`

Running JavaScript

What JavaScript is and where it runs

JavaScript is the programming language of the web: browsers run it to make pages react to the user, and Node.js runs the same language outside the browser.

Dynamically typed

You never write a type: the value decides it while the program runs, and a variable can hold a number now and a string later.

Single-threaded

One statement runs at a time: while a long loop runs, the page cannot react to clicks (see [Blocking and non-blocking](#)).

Interpreted and JIT-compiled

There is no separate compile step for you: the engine translates frequently used code into machine code on the fly (**Just-In-Time**) to make it fast.

Three ways to run it while you learn

- **The browser console:** press F12 and open the Console tab.
- **A page:** a `<script>` tag in an HTML file.
- **A terminal:** `node file.js`.

```
console.log('Hello from JavaScript');
console.log(2 + 3);
console.log(typeof 42);
console.log(typeof document); // no page here, like Node
```

Example. Type `2 + 3` in the browser console and it answers 5; save `console.log(2 + 3);` in `sum.js` and run `node sum.js`: the terminal prints 5. Now try `typeof document`: the browser console says 'object', Node says 'undefined'. Same language, different environment. The Try it box runs in a background worker with no page, so it behaves like Node.

Common mistake. Assuming `document` exists everywhere. Running a page script with `node app.js` stops with `ReferenceError: document is not defined`: the code is fine, but Node has no page to give it.

The console: `console.log` and `console.assert`

The console is the developer's output window: `console.log` prints values, and `console.assert` prints a message only when a check fails.

`console.log(a, b, c)`

Prints several values separated by spaces. Strings appear without quotes; arrays and objects are shown with their contents.

`console.error(x)`

Prints in red (and to the error stream in Node).

`console.assert(condition, message)`

A tiny automatic test: when the condition is true it prints **nothing**; when it is false it prints `Assertion failed: message`. A silent console means every check passed.

```
const price = 4;
const total = price * 2;
// total is 8
console.log('total is', total);

// silent: passed
console.assert(total === 8, 'double 4');
// Assertion failed: times zero
console.assert(price * 0 === 1, 'times zero');
```

```
const price = 20;
const quantity = 3;
console.log('price:', price, 'quantity:', quantity);
console.log('total', price * quantity);
console.log(['a', 'b'], { item: 'pen', price: 2 });
console.error('errors print in red');
```

Example. Converting 100 °C to Fahrenheit with `const f = 100 * 9 / 5 + 32;` add `console.assert(f === 212, 'boiling')` under it. An empty console means the check passed; if a later edit breaks the formula, Assertion failed: boiling tells you exactly which check to look at.

Common mistake. Thinking that printing a value is the same as producing it. `console.log` shows a value **to you**; it does not give the value back to the code that asked for it. A function that only logs its result returns undefined (see [Functions](#)).

Reading error messages

An error message names the kind of problem, describes it and points to the file, line and column where JavaScript had to stop.

The three errors you will meet most (messages as Chrome and Node word them)

Type	Meaning	Typical message	Typical cause
SyntaxError	The code cannot even be read; nothing in the file runs	Unexpected token '}'	A missing or extra bracket, quote or comma
ReferenceError	A name does not exist here	totl is not defined	A typo, a variable used outside its scope, or a <code>let/const</code> used before its line
TypeError	The value exists but is the wrong kind for the operation	Cannot read properties of undefined (reading 'name'), x is not a function, Assignment to constant variable.	Reading a property of undefined/null, calling something that is not a function, reassigning a const

```
try {
  const user = undefined;
  console.log(user.name);
} catch (err) {
  console.log(err.name);
  // → TypeError
}

try {
  // typo for total
  console.log(totl);
} catch (err) {
  console.log(err.name);
  // → ReferenceError
}
```

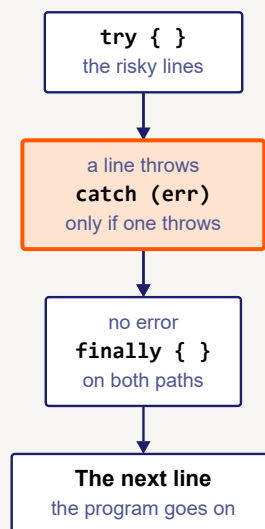
```
const list = [];
console.log(0 / list.length); // NaN: no error, but a logic bug
```

Example. A function `average(list)` is tested with `[]` and prints `NaN` instead of `0`. There is no error, so it is a logic bug. `NaN` is the clue: it comes from `0 / 0`, so the sum (`0`) was divided by the length (`0`). The fix is to handle the empty list before dividing.

Common mistake. Fixing the line the error points at when the real cause is upstream. `TypeError: Cannot read properties of null on line 20` usually means the variable became `null` earlier, for example because a lookup with a misspelt name found nothing. Also: fix the **first** error in the console first; later ones are often consequences of it.

try and catch: handling errors

`try { ... } catch (err) { ... }` runs code that might fail and, if it throws, jumps to the `catch` block with the error instead of stopping the program.



The error jumps to catch; finally runs on both paths.

<code>catch (err)</code>	Receives the error object: <code>err.name</code> (<code>TypeError...</code>), <code>err.message</code> (the description) and <code>err.stack</code> (where it happened).
<code>finally { }</code>	Optional. Runs after try or catch, whether or not there was an error: the place for clean-up.
<code>throw new Error('...')</code>	Raises your own error, with your message, when your code finds a value it cannot work with.
<code>throw err inside catch</code>	Rethrows: hands on an error you cannot deal with here, after logging it, to code further out.

```
try {
  const user = undefined;
  // throws a TypeError
  console.log(user.name);
  console.log('never printed');
} catch (err) {
  console.log(err.name);
  // → TypeError
  // Cannot read properties of undefined (reading 'name')
  console.log(err.message);
} finally {
  console.log('finally runs either way');
}

try {
  throw new Error('Price must be positive');
} catch (err) {
```

```
// Price must be positive
console.log(err.message);
}
console.log('the program keeps going');
```

Example. Reading a property of a missing value: `const user = undefined; console.log(user.name);` stops the whole script with a `TypeError`. Inside `try { ... } catch (err) { console.log(err.message); }`, the script prints `Cannot read properties of undefined (reading 'name')` and keeps going.

Common mistake. An empty catch: `catch (err) {}`. The error disappears, the program goes on with a wrong or missing value, and nothing tells you where it broke. At least `log err.message`; if this code cannot fix the problem, rethrow it with `throw err`.

Variables, types and values

Declaring variables: let, const and var

A variable is a name bound to a value: a `const` name can never be pointed at another value, a `let` name can, and `var` is the older keyword with looser rules.

The three keywords side by side

Keyword	Reassign?	Scope	Used before its line	Declare the same name twice
<code>const</code>	No	Block { }	<code>ReferenceError</code>	<code>SyntaxError</code>
<code>let</code>	Yes	Block { }	<code>ReferenceError</code>	<code>SyntaxError</code>
<code>var</code>	Yes	Whole function (ignores blocks)	Silently undefined	Allowed, silently

```
let count = 0;
count = count + 1;
console.log(count);
// → 1

const city = 'Madrid';
try {
  city = 'Bilbao';
} catch (err) {
  console.log(err.name);
  // → TypeError
}

const items = [];
// allowed: changes the array
items.push('pen');
console.log(items);
```

Example. A shopping cart: `const TAX_RATE = 0.21;` never changes, so `const`. `let total = 0;` grows as items are added (`total = total + price;`), so `let`. `const items = [];` is also `const`: you will add items **into** the array, but the name `items` always points at the same array.

Common mistake. "`const` means the value can never change." It only freezes the **binding** (which value the name points at). `const items = [];` `items.push('pen');` is legal, because the array is changed, not replaced. See [Values vs references](#).

Data types and typeof

Every value has a type: JavaScript has seven primitive types plus objects, and the `typeof` operator tells you which one a value is, with two famous quirks.

undefined "No value", set by the engine: a declared variable with no value, a missing property, a missing argument, the result of a function without return.

null "No value", written by a programmer on purpose: "intentionally empty".

What typeof answers

Value	typeof value	Note
42, 3.14, NaN	'number'	One type for integers and decimals; even "Not a Number" is a number
'hi', ''	'string'	
true	'boolean'	
undefined	'undefined'	Also for a name that was never declared
null	'object'	A historical bug kept for compatibility: test with value === null
[1, 2], { a: 1 }	'object'	Arrays are objects: test with Array.isArray(value)
function () {}	'function'	Functions are objects too, but typeof reports them separately

```
console.log(typeof 42, typeof 'hi', typeof true);
console.log(typeof undefined, typeof null);
console.log(typeof [1, 2], Array.isArray([1, 2]));
console.log(typeof function () {});
```

```
let x = 5;
// the value decides the type
x = 'five';
console.log(typeof x);
```

Example. Text typed by a user, or read from a file, always arrives as a **string**: if the user types 42 into an age box, the program receives '42', so typeof gives 'string' and '42' + 1 is '421'. Converting with Number('42') gives the number 42.

Common mistake. Testing typeof x === 'object' to mean "x is a real object". null passes that test too, and the next line x.name then crashes with a TypeError. Check x !== null as well.

Strings and template literals

A string is an unchangeable sequence of characters; template literals (backticks) can embed expressions with \${...} and span several lines.

String tools you will use most, for s = 'Riverside FC'

Expression	Result	What it does
s.length	12	Number of characters (a property, no parentheses)
s[0], s[s.length - 1]	'R', 'C'	One character by position
s.toLowerCase() / s.toUpperCase()	'riverside fc' / 'RIVERSIDE FC'	Case conversion (a new string)
s.includes('FC')	true	Does it contain this text? (case-sensitive)
s.indexOf('z')	-1	Position of the first match, -1 when absent
s.slice(0, 9)	'Riverside'	Characters from position 0 up to (not including) 9
' hi '.trim()	'hi'	Removes spaces at both ends (useful on user input)
'a,b,c'.split(',')	['a', 'b', 'c']	Cuts a string into an array (see Arrays)
['a', 'b'].join('-')	'a-b'	The opposite: glues an array into a string

```
const club = 'Riverside FC';
console.log(club.length);
console.log(club[0], club.toUpperCase());
console.log(club.includes('FC'), club.indexOf('z'));
console.log(club.slice(0, 9));
console.log('a,b,c'.split(','));

const goals = 3;
console.log(`${club} scored ${goals} goal${goals === 1 ? '' : 's'}`);

// result thrown away...
club.toLowerCase();
// ...so club is unchanged
console.log(club);
```

Example. Building a score line: the template literal `${home} ${homeGoals}-${awayGoals} ${away}`, written between backticks, with `home = 'Riverside'`, `homeGoals = 2`, `awayGoals = 1`, `away = 'Hillside'` gives `Riverside 2-1 Hillside`. Any expression fits inside `${...}`, even the ternary `${homeGoals > awayGoals ? 'win' : 'no win'}` (see [Conditionals](#)).

Common mistake. Expecting a method to change the string: after `club.toUpperCase()`; the variable `club` is unchanged. Strings cannot be changed, so keep the result: `const loud = club.toUpperCase()`; A second classic: writing `'Hello ${user}'` with normal quotes, which prints the `${user}` literally; interpolation only works between backticks.

Numbers, arithmetic and the remainder operator

JavaScript has a single number type for integers and decimals; besides `+` `-` `*` `/` it has `%` (remainder) and `**` (power), and decimals are stored in binary, so some results are slightly off.

x += 5, x++ Shorthand: `x = x + 5` (also `-=`, `*=`, `/=`); `x++` adds 1, `x--` subtracts 1.

Math `Math.round(x)`, `Math.floor(x)` (down), `Math.ceil(x)` (up), `Math.sqrt(x)`, `Math.abs(x)`. To cents: `Math.round(x * 100) / 100`. `x.toFixed(2)` also rounds, but returns a **string**.

Number(text) `Number('42')` is 42; `Number('abc')` is NaN ("Not a Number", the result of failed maths). NaN equals nothing, not even itself: test it with `Number.isNaN(x)`.

Division by zero Does not crash: `1 / 0` is Infinity.

```
console.log(17 % 5);
// → 2
console.log(10 % 2 === 0);
// → true: 10 is even
console.log(0.1 + 0.2);
console.log(Math.round(12.3456 * 100) / 100);
console.log(Number('42') + 1, Number('abc'));
console.log(Number('abc') === NaN, Number.isNaN(Number('abc')));

let score = 10;
score += 5;
score++;
console.log(score);
// → 16
```

Example. Converting 135 minutes into hours and minutes: `Math.floor(135 / 60)` is 2 whole hours and `135 % 60` is 15 minutes left over, so for `m = 135` the template literal `${Math.floor(m / 60)} h ${m % 60} min` (between backticks) prints `2 h 15 min`.

Common mistake. Testing `x === NaN` to detect a failed conversion. It is **always** false, even when `x` is NaN, because NaN never equals anything. Use `Number.isNaN(x)`.

Type coercion and == vs ===

Coercion is JavaScript converting a value to another type automatically; `===` compares type and value without converting, while `==` converts first, so the safe default (and the one used on this site) is `===`.

Strict equality === True only when both sides have the same type **and** the same value; it never converts. Use `===` and `!==` always.

Loose equality == Converts the two sides by a long rule table, then compares. The results are hard to predict and not even consistent: `0 == ''` and `0 == '0'` are both true, yet `'' == '0'` is false.

Loose versus strict equality

Comparison	==	===
'5' and 5	true	false

Comparison	==	===
0 and ''	true	false
0 and false	true	false
null and undefined	true	false
'' and '0'	false	false
NaN and NaN	false	false

```
console.log('5' + 1);
// → '51': + joins text
console.log('5' - 1);
// → 4: - converts to number
console.log('5' == 5, '5' === 5);
console.log(0 == '', 0 === '');
console.log(null == undefined, null === undefined);
console.log(NaN == NaN);
// two different arrays
console.log([1] === [1]);
```

Example. An age gate reads `const raw = '18';` from a form. `raw >= 18` is true (coerced), `raw === 18` is false (string vs number), and `Number(raw) === 18` is true. Converting once, at the point where the input enters your code, makes every later comparison predictable.

Common mistake. Comparing text from a form with a number: `if (age === 18)` is never true while `age` is the string `'18'`. Switching to `==` makes it "work" but hides the real problem; convert with `Number()` instead.

Truthy and falsy values

In a condition every value counts as true or false: the falsy values are `false`, `0`, `-0`, `0n`, `''`, `null`, `undefined` and `NaN`; everything else is truthy.

- **See the verdict:** `Boolean(value)` (or `!!value`) shows the true/false JavaScript will use.
- **Short tests:** `if (name)` rejects both an empty and a missing name; `if (list.length)` means "the list is not empty".

```
const cases = [
  ['0', 0], ['', ''], [' ' ' ', ' '], ['0', '0'],
  ['null', null], ['undefined', undefined], ['NaN', NaN],
  ['[]', []], ['{}', {}], ['-1', -1],
];
for (const pair of cases) {
  console.log(pair[0], Boolean(pair[1]) ? 'truthy' : 'falsy');
}
```

Example. A search box: `const query = text.trim(); if (!query) { console.log('Type something'); }`. An empty or all-spaces input becomes `''`, which is falsy, so one test covers both cases.

Common mistake. Using truthiness when `0` or `''` is a valid value: `if (stock) { ... } else { ... 'no data' }` says "no data" when the stock is exactly 0. Test the case you really mean, for example `stock !== undefined`. The opposite trap: `if (results)` is always true for an array, even an empty one; test `results.length`.

Logic and control flow

Logical operators and short-circuiting

`&&`, `||` and `??` combine values and stop as soon as the answer is known, returning one of their operands; `!` turns any value into the opposite boolean.

a && b If `a` is falsy, the answer is `a`; otherwise it is `b`.

a || b If `a` is truthy, the answer is `a`; otherwise it is `b`.

a ?? b Nullish coalescing: if `a` is `null` or `undefined`, the answer is `b`; otherwise `a`.

!a Always a real boolean: `!'hello'` is false, so `!!value` converts any value to true/false.

Defaults: `||` versus `??`

Expression	Result	Why
<code>'' 'guest'</code>	<code>'guest'</code>	<code>''</code> is falsy
<code>'' ?? 'guest'</code>	<code>''</code>	<code>''</code> is not null/undefined
<code>0 50</code>	<code>50</code>	<code>0</code> is falsy
<code>0 ?? 50</code>	<code>0</code>	<code>0</code> is a real value
<code>undefined ?? 50</code>	<code>50</code>	No value at all

```
console.log(true && 'yes');
// → 'yes'
console.log(0 && 'never');
// → 0
console.log('' || 'guest');
// → 'guest'
console.log(0 || 50, 0 ?? 50);
// → 50 0
console.log(null ?? 'default');
console.log(!'hello', !!'hello');
```

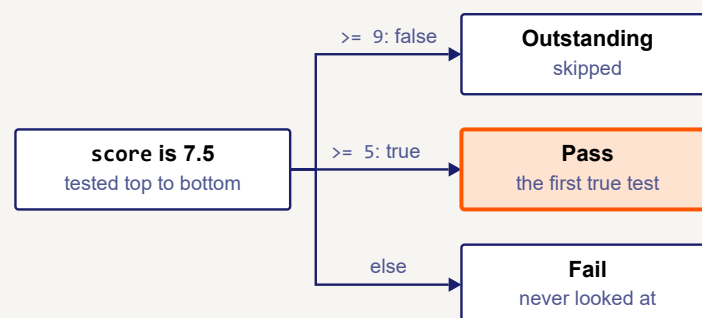
```
function shout() {
  console.log('shout ran');
  return true;
}
// shout never runs
console.log(false && shout());
const user = undefined;
console.log(user && user.name);
// → undefined, no crash
```

Example. A greeting: `const display = nickname || fullName || 'guest'`; picks the first non-empty name. A volume setting: `const volume = settings.volume ?? 50`; keeps a deliberate volume of 0, which `||` would wrongly replace by 50.

Common mistake. Using `||` for a default when 0, '' or false is a legitimate value. `const volume = settings.volume || 50` turns a muted player (volume 0) into volume 50. Use `??` when only "missing" should trigger the default.

Conditionals: if/else, the ternary and switch

Conditionals choose which code runs: `if` / `else if` / `else` for general branching, the ternary `cond ? a : b` to pick one of two values, and `switch` to compare one value against fixed cases.



The conditions are tested top to bottom; the first true one wins.

Ternary `cond ? a : b` An **expression**: it produces a value you can store or print. Use it to choose a value, not to run long actions, and avoid nesting ternaries.

switch (value) Compares the value against each case with **strict equality** (`===`). Execution starts at the matching case and **falls through** into the next ones until a

break; default runs when no case matches. Stacking cases without break (case 'Sat': case 'Sun':) is the one useful form of fall-through.

- **Comparison operators:** ==, !=, <, >, <=, >=. A single = is assignment, not comparison: if (x = 5) assigns 5 and is always truthy.
- **Guard clause:** inside a function, an early return at the top for the bad case avoids deep nesting.

```
const score = 7.5;
if (score >= 9) {
  console.log('Outstanding');
} else if (score >= 5) {
  console.log('Pass');
} else {
  console.log('Fail');
}

const label = score >= 5 ? 'pass' : 'fail';
console.log(label);

const day = 'Sun';
switch (day) {
  case 'Sat':
  case 'Sun':
    console.log('weekend');
    break;
  default:
    console.log('weekday');
}

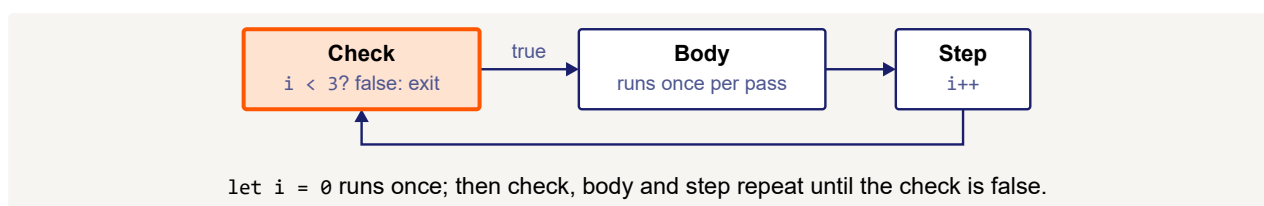
const level = '2';
switch (level) {
  case 2:
    console.log('number 2');
    break;
  default:
    console.log('no match: switch uses ===');
}
```

Example. Exam labels: if (score >= 9) label = 'Outstanding'; else if (score >= 5) label = 'Pass'; else label = 'Fail';. With score = 7.5 the first test fails, the second passes, and the else is never looked at.

Common mistake. Ordering branches from general to specific: if (score >= 5) { ... } else if (score >= 9) { ... }. A 9.5 already matches the first test, so the "Outstanding" branch can never run.

Loops: for, while, break and continue

A loop repeats a block while a condition stays truthy; for puts the start, the condition and the step on one line, while only has the condition.



break Leaves the loop immediately.

continue Skips the rest of this pass and goes to the next check.

return Inside a function, leaves the loop **and** the whole function.

- **Accumulator pattern:** create the result before the loop (let total = 0), update it inside, use it after the loop.

- **Off by one:** `<` versus `<=` decides whether the last value is included. Check the first and the last pass explicitly.

```
for (let i = 0; i < 3; i++) {
  console.log('pass', i);
}

let n = 40;
let steps = 0;
while (n > 1) {
  n = Math.floor(n / 2);
  steps++;
}
console.log('halvings:', steps);

for (let i = 1; i <= 10; i++) {
  // skip non-multiples
  if (i % 3 !== 0) continue;
  // stop completely
  if (i > 7) break;
  console.log('multiple of 3:', i);
}
```

Example. How many times can you halve 40 before reaching 1? `let n = 40, steps = 0; while (n > 1) { n = Math.floor(n / 2); steps++; }`. Trace: 40, 20, 10, 5, 2, 1, so steps ends at 5. You could not know "5" before running, which is why while fits.

Common mistake. An off-by-one count: `for (let i = 1; i < 10; i++)` runs 9 times, not 10, because the pass with `i = 10` fails the check. Decide whether the end is included (`<=`) or not (`<`), and check the first and the last pass by hand.

Arrays and objects

Arrays: indexes, length, push and pop

An array is an ordered list of values under one name; each value is read by its position, its **index**, which starts at 0.

`push(x) / pop()` Add at the end / remove the last element and return it.

`unshift(x) / shift()` The same at the start.

`arr[i] = value` Replaces one element.

- **Any mix of values**, including objects: an **array of objects** (a list of records, see [Objects](#)) is the everyday shape of data in pages and in the JSON returned by almost every API.
- **Arrays are objects:** `typeof []` is `'object'`, so recognise one with `Array.isArray(x)`.
- **A const array can still change** (push, pop, edit): `const` only stops the name from pointing at another array (see [Values vs references](#)).

```
const students = ['Anna', 'Bob', 'Joan'];
console.log(students[0], students.length);
// last
console.log(students[students.length - 1]);
console.log(students[10]);
// → undefined, no error

students.push('Zoe');
const removed = students.pop();
console.log(removed, students.length);

students[1] = 'Bea';
console.log(students);
```

```
const squad = [{ name: 'Elena', pos: 'GK' }, { name: 'Sofia', pos: 'MF' }];
console.log(squad[1].name);
```

Example. A squad as an array of objects: `const squad = [{ name: 'Elena', pos: 'GK' }, { name: 'Sofia', pos: 'MF' }];`. `squad.length` is 2, `squad[1].name` is 'Sofia', and `squad.push({ name: 'Ines', pos: 'DF' })` adds a third player.

Common mistake. Reading the last element with `arr[arr.length]`. Because indexes start at 0, that position is one past the end and gives undefined; the last element is `arr[arr.length - 1]`. The same slip in a loop, for `(let i = 0; i <= arr.length; i++)`, reads that missing element on its last pass: use `i < arr.length`.

Objects: properties, dot and bracket notation

An object groups named values, its **properties**, under one variable; you read and write them with dot notation `obj.key` or bracket notation `obj['key']`.

Dot notation `player.name` For names you know while writing the code.

Bracket notation `player[key]` For names held in a variable, or keys with spaces or dashes (`obj['shirt-size']`).

A missing property Reading it gives undefined; reading a property of undefined throws a `TypeError`. That is why `user.address.city` can crash when a level is missing; `user.address?.city` gives undefined instead.

- **Add and remove:** assigning a new key adds it (`player.club = 'Riverside'`); delete `player.club` removes it.
- **Count:** `Object.keys(obj)` returns the keys as an array, so `Object.keys(obj).length` counts them.
- **Order:** keys follow insertion order (integer-like keys come first, ascending). If order is part of your data, use an array.

Array or object?

	Array	Object
Look values up by	position (0, 1, 2...)	name ('title', 'price')
Best for	a list of similar items	one item with named fields
Literal	[]	{ }
Count	<code>arr.length</code>	<code>Object.keys(obj).length</code>

```
const player = {
  name: 'Sofia',
  number: 10,
  stats: { goals: 7, assists: 3 },
  // method shorthand
  celebrate() {
    return 'Goal!';
  },
};

console.log(player.name, player['number']);
const field = 'assists';
console.log(player.stats[field], player.stats.field);
console.log(player.club);
// → undefined: missing key

player.club = 'Riverside';
console.log(Object.keys(player));
console.log(player.celebrate());
```

Example. A player record: `const player = { name: 'Sofia', number: 10, stats: { goals: 7, assists: 3 } };`. `player.stats.goals` is 7. To read whichever stat the user chose, `const field = 'assists'; player.stats[field]` gives 3.

Common mistake. Using dot notation with a variable: `player.stats.field` looks for a property literally called "field" and gives undefined. When the key is in a variable, use brackets: `player.stats[field]`.

for...of, for...in and looping over strings

`for...of` walks the **values** of an array or the characters of a string; `for...in` walks the **keys** of an object and should not be used on arrays.

for (const item of collection) "For each thing in this collection." No index to manage, so no off-by-one to get wrong: the modern default for arrays. It works on anything **iterable**: arrays and strings (one character per pass). A plain object is **not** iterable: `for...of` on it throws a `TypeError`.

for (const key in object) Gives each **property name**, as a string; read the value with `object[key]`. On an array it gives the indexes **as strings** (`'0'`, `'1'...`) and may also include extra properties, so do not use it there.

Which loop for which job

You need...	Use
Every value of an array or every character of a string	<code>for (const x of arr)</code>
The index as well, or a custom step	<code>for (let i = 0; i < arr.length; i++)</code>
Every key of an object	<code>for (const key in obj)</code> Or <code>Object.keys(obj)</code>
Repeat until something happens	<code>while (condition)</code>

```
const tags = ['html', 'css', 'js'];
for (const tag of tags) {
  console.log('value:', tag);
}
for (const i in tags) {
  console.log('key:', i, typeof i);
}

const player = { name: 'Sofia', number: 10 };
for (const key in player) {
  console.log(key, '=', player[key]);
}

for (const ch of 'hey') {
  console.log(ch);
}

try {
  for (const x of player) {}
} catch (err) {
  console.log(err.name);
  // → TypeError: objects are not iterable
}
```

Example. Counting capital letters in 'Hello World': `let caps = 0; for (const ch of 'Hello World') { if (ch !== ch.toLowerCase()) caps++; }` gives 2. No index, no length, no off-by-one.

Common mistake. Using `for...in` on an array and doing arithmetic with the key: `for (const i in tags) console.log(i + 1)` prints 01, 11, 21, because `i` is the **string** `'0'` and `+` joins text.

Values vs references: copying and mutation

Assigning a primitive copies the value itself; assigning an object or array copies a **reference** to the same data, so a change made through one variable is visible through the other.

Mutation Changing the object (`list.push(x)`, `obj.key = v`): every variable pointing at it sees the change.

Reassignment Pointing one name at something else (`b = [ ]`): only that name moves. That is why `const` does not freeze an array: it forbids reassignment, not mutation.

- **An independent copy:** `[...arr]` for arrays, `{ ...obj }` for objects (the spread syntax).
- **Shallow:** arrays or objects nested inside are still shared; `structuredClone(obj)` makes a deep copy.
- **Equality follows the same model:** `===` on objects asks "same address?", so `[1] === [1]` is false.

```
let a = 5;
let b = a;
b = 6;
console.log(a, b);
// → 5 6: independent

const list = ['x'];
// same array
const alias = list;
alias.push('y');
console.log(list);

// a new array
const copy = [...list];
copy.push('z');
console.log(list.length, copy.length);

console.log([1] === [1], list === alias);

const team = { name: 'A', players: ['Ana'] };
// shallow copy
const clone = { ...team };
clone.players.push('Bea');
// nested array is shared
console.log(team.players);
```

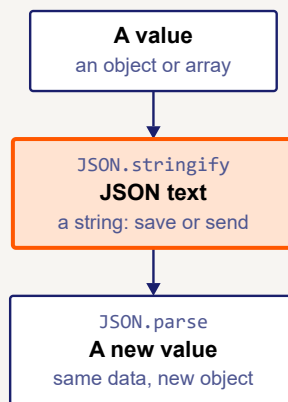
Example. A function receives a reference too. `function addBonus(scores) { scores.push(10); }` called with `const mine = [7, 8]` leaves `mine` as `[7, 8, 10]` after the call: the function changed the caller's array. Writing `return [...scores, 10];` instead returns a new array and leaves `mine` intact.

Common mistake. "I copied the array, so editing the copy is safe" after writing `const backup = list;` That copies only the reference: `backup.push(...)` changes `list` too. Use `const backup = [...list];`.

JSON: data as text

JSON (JavaScript Object Notation) is a text format for data: `JSON.stringify(value)` turns a value into JSON text,

and `JSON.parse(text)` turns the text back into a new value.



JSON is the text form of your data; parsing it builds a new copy.

What survives a round trip, stringify then parse

In the value	After the trip
Numbers, strings, true/false, null, arrays, plain objects	The same data, in a new object
A property set to undefined, a function	Dropped
A Date	A string such as '1970-01-01T00:00:00.000Z'
NaN, Infinity	null

`JSON.stringify(value, null, 2)` Indents the text by two spaces: readable, for files and logs.

`JSON.parse(text)` on bad text Throws a `SyntaxError`. Text from outside your program can be broken, so parse it inside try and catch.

```

const task = { id: 1, title: 'Buy milk', done: false, tags: ['home'] };
const text = JSON.stringify(task);
console.log(text);
// → {"id":1,"title":"Buy milk","done":false,"tags":["home"]}
// string
console.log(typeof text);

const back = JSON.parse(text);
// Buy milk false: a new object
console.log(back.title, back === task);

console.log(JSON.stringify({ a: undefined, f() {}, when: new Date(0) }));
// {"when":"1970-01-01T00:00:00.000Z"}

try {
  // not JSON: keys and strings need double quotes
  JSON.parse("{ title: 'x' }");
} catch (err) {
  console.log(err.name);
  // → SyntaxError
}
  
```

Example. A task list saved in a file or sent to a server travels as JSON text: `JSON.stringify(tasks)` before it leaves, `JSON.parse(text)` when it comes back. The receiver gets the same data in new objects, so changing them never touches the sender's.

Common mistake. Treating JSON text as an object: after `const text = JSON.stringify(task);`, `text.title` is undefined, because `text` is a string. Parse it first: `JSON.parse(text).title`. The reverse slip: `'Task: ' + task` gives `'Task: [object Object]'`; stringify the object to see its data.

Functions and scope

Functions: parameters, arguments and return

A function is a named, reusable block of code: it receives inputs through its **parameters**, runs, and hands one result back with `return`.



A call hands values in; return hands one value back.

- return value** Does two things at once: **sends the value back** to the place where the function was called, and **ends the function**; any line after it is skipped. A function that ends without `return` gives back `undefined`.
- Argument count** Not checked: a missing argument is `undefined` inside the function, an extra one is ignored.
- Pure function** Its result depends only on its arguments and it changes nothing outside: the easiest kind to test with `console.assert`.

```

function area(width, height) {
  return width * height;
}

const a = area(3, 4);
console.log(a);
// → 12
console.log(area(3, 4) + area(1, 2));
console.log(area(3));
// → NaN: height is undefined

function logOnly(x) {
  console.log('inside:', x * 2);
}
const r = logOnly(5);
console.log('returned:', r);
// → undefined

// without () you get the function itself
console.log(typeof area);
  
```

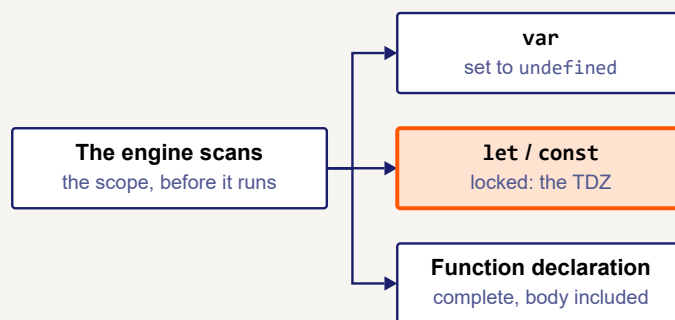
Example. `function area(width, height) { return width * height; }`. `const a = area(3, 4);` stores 12; `area(3, 4) + area(1, 2)` is 14, because each call is replaced by its returned value. `area(3)` gives NaN: height is undefined and `3 * undefined` is not a number.

Common mistake. Printing instead of returning. `function total(a, b) { console.log(a + b); }` shows the right number in the console, yet `total(2, 3)` **is** `undefined`, so `total(2, 3) * 2` is NaN and any test of its result fails. Return the value; let the caller decide whether to print it.

Scope, hoisting and the temporal dead zone

Scope is the region of code where a name is visible; hoisting is the engine registering every declaration before

it runs a scope, so **var** reads as undefined early while **let** and **const** throw an error.



Every declaration is hoisted; only **let** and **const** are locked until their line.

Block scope

- **A block** is any pair of braces: `if`, `for`, `while` or a bare `{ }`.
- **let** and **const** live only inside the block where they are declared; **var** ignores blocks and leaks out to the enclosing function (or the whole script).
- **Nested scopes look outward**: inner code can read outer names, but outer code never sees inner ones.
- **Shadowing**: an inner `let x` hides an outer `x` inside its block; the outer one is untouched.
- **Loops**: `for (let i = ...) { ... }` gets a fresh `i` per pass that disappears after the loop; with `var i` the counter survives the loop.

```

console.log(a);
// → undefined: var is hoisted with undefined
var a = 1;

try {
  // b is in its temporal dead zone
  console.log(b);
} catch (err) {
  console.log(err.name); // ReferenceError
}
let b = 2;

if (true) {
  var leaky = 'var escapes the block';
  let tidy = 'let stays inside';
}
console.log(leaky);
console.log(typeof tidy); // 'undefined': not visible out here

for (var i = 0; i < 3; i++) {}
console.log(i);
// → 3: the var counter survived the loop
  
```

Example. Shadowing plus TDZ: `let x = 'outer'; { console.log(x); let x = 'inner'; }` throws `ReferenceError` instead of printing `outer`. The inner `let x` was hoisted to the top of its block and is in its TDZ at the `console.log` line, which proves **let** is hoisted, just locked.

Common mistake. "let and const are not hoisted." They are; they are just unusable until their line. The visible difference with `var` is the error versus the silent undefined, not whether hoisting happens.

Declarations, expressions and arrow functions

A function can be written as a **declaration** (`function f() {}`), as an **expression stored in a variable** (`const f = function () {}`), or as an **arrow function** (`const f = () => ...`).

The three forms of the same function

Form	Syntax	Callable before its line?	Typical use
Declaration	<code>function add(a, b) { return a + b; }</code>	Yes (hoisted)	Named helpers at the top level of a file
Expression	<code>const add = function (a, b) { return a + b; };</code>	No: ReferenceError	Storing or passing a function as a value
Arrow	<code>const add = (a, b) => a + b;</code>	No: ReferenceError	Short helpers and callbacks, functions passed to other functions (see Functions as values)

Arrow rules

- **One parameter:** the parentheses are optional (`n => n * 2`); the code on this site keeps them for consistency.
- **Concise body** (no braces): returns its expression automatically, (`n`) `=> n * 2`.
- **Block body** (braces): needs an explicit `return`.
- **Returning an object literal** from a concise body: wrap it in parentheses, `() => ({ ok: true })`, otherwise the braces are read as a block.

```
console.log(early(2));
function early(x) {
  return x + 1;
}

try {
  late(2);
} catch (err) {
  console.log(err.name);
  // → ReferenceError: const not ready yet
}

const late = function (x) {
  return x + 1;
};

const square = (n) => n * n;
const squareBraces = (n) => { n * n; };
console.log(square(4), squareBraces(4));

const makeUser = (name) => ({ name: name, active: true });
console.log(makeUser('Ana'));
```

Example. `const toEuros = (cents) => cents / 100;` (concise body, implicit return) and `const toEuros = (cents) => { return cents / 100; };` (block body) are equivalent; `toEuros(1250)` is 12.5 in both.

Common mistake. Adding braces to an arrow function and forgetting `return`: `const square = (n) => { n * n; };` makes `square(4)` return undefined. Either remove the braces or write `return n * n;`.

Default parameters

A default parameter (`function f(x = 10)`) **supplies the value to use when the caller passes nothing, or passes undefined, for that parameter.**

```
function shippingCost(weightKg, ratePerKg = 4) {
  return weightKg * ratePerKg;
}

console.log(shippingCost(2));
// → 8
console.log(shippingCost(2, 6));
// → 12
console.log(shippingCost(2, undefined)); // 8
console.log(shippingCost(2, null));
// → 0
console.log(shippingCost(2, 0));
// → 0: free shipping kept
```

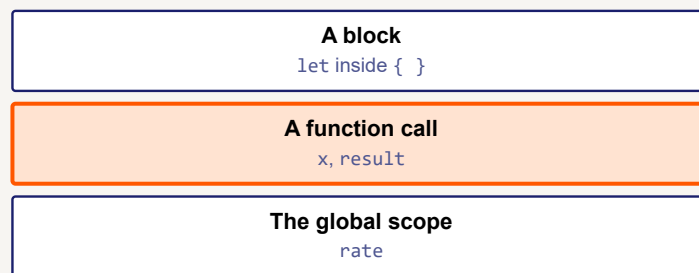
```
function box(w, h = w) {
  return w + ' x ' + h;
}
console.log(box(3), box(3, 5));
```

Example. Shipping: `function shippingCost(weightKg, ratePerKg = 4) { return weightKg * ratePerKg; }`. `shippingCost(2)` is 8 (default used), `shippingCost(2, 6)` is 12, `shippingCost(2, undefined)` is 8, and `shippingCost(2, null)` is 0, because `null` is a value and `2 * null` is 0.

Common mistake. Using the old pattern `rate = rate || 4` inside the function. It also replaces a legitimate 0 (free shipping) by 4. A default parameter only replaces undefined.

Function scope and functions as values

Variables and parameters declared inside a function exist only inside it, for the duration of each call; and functions are values that can be stored, passed to other functions and returned.



- **Prefer** return over changing outer variables from inside a function: it keeps functions pure and testable.
- **Pass the name, not a call:** `applyTwice(double, 3)` hands over the function; `applyTwice(double(), 3)` calls it immediately and hands over its result.

```
const rate = 2;
function scale(x) {
  // reads the outer rate
  const result = x * rate;
  return result;
}
console.log(scale(5));
// → 10
console.log(typeof result);
// → 'undefined': local to scale

function applyTwice(fn, value) {
  return fn(fn(value));
}
console.log(applyTwice((n) => n + 3, 1));
console.log(applyTwice((s) => s + '!', 'hi'));

const tools = [Math.floor, Math.ceil];
// a function stored in an array
console.log(tools[1](2.1));
```

Example. `function applyTwice(fn, value) { return fn(fn(value)); }` receives a function as its first argument. `applyTwice((n) => n + 3, 1)` computes $(1 + 3) + 3 = 7$; `applyTwice((s) => s + '!', 'hi')` gives 'hi!!'.

Common mistake. Expecting to read a function's local variable from outside: after function `scale(x) { const result = x * 2; return result; }`, writing `console.log(result)` at the top level is a `ReferenceError`. Use the returned value: `const r = scale(5);`.

Array methods: forEach, map, filter, find, includes

Array methods run a function you give them (a **callback) on each element:** `forEach` **does something**, `map` **transforms**, `filter` **keeps some**, `find` **returns the first match**, and `includes` **checks membership**.

What each method returns (none of them changes the original array)

Method	The callback returns...	The method returns...
<code>arr.forEach(fn)</code>	nothing useful	undefined: use it for an action per element
<code>arr.map(fn)</code>	the new value for this element	a new array , same length
<code>arr.filter(fn)</code>	true/false: keep it?	a new array , possibly shorter or empty
<code>arr.find(fn)</code>	true/false: is this the one?	the first match, or undefined
<code>arr.includes(value)</code>	(no callback)	true or false, comparing with ===

```
const prices = [12, 45, 8, 30];

prices.forEach((p) => console.log('price', p));
console.log(prices.map((p) => p * 2));
console.log(prices.filter((p) => p < 20));
console.log(prices.find((p) => p > 40));
console.log(prices.find((p) => p > 100));
// → undefined
console.log(prices.includes(8), prices.includes('8'));
// unchanged
console.log(prices);

console.log(prices.map((p) => { p * 2; })); // missing return
```

Example. Prices [12, 45, 8, 30]: `prices.map((p) => p * 2)` gives [24, 90, 16, 60]; `prices.filter((p) => p < 20)` gives [12, 8]; `prices.find((p) => p > 40)` gives 45; `prices.includes(8)` is true but `prices.includes('8')` is false (string versus number). `prices` itself is still [12, 45, 8, 30].

Common mistake. Writing a `map` callback with braces but no return: `prices.map((p) => { p * 2; })` gives [undefined, undefined, undefined, undefined]. With braces, an arrow function needs an explicit return; without braces (`(p) => p * 2`) the value is returned automatically. Similarly, `const doubled = prices.forEach(...)` is always undefined: `forEach` returns nothing.